

Structure de données

Les pointeurs

DR. HASNA ELALAOUI

h.elalaoui@uca.ac.ma

POINTEURS

Détails sur la partie

- ❑ Contexte général
- ❑ Notion de pointeurs
- ❑ Manipulation des pointeurs
- ❑ Allocation dynamique de la mémoire

Contexte Général

Rappel sur la notion de variable

❑ Une **variable** est une petite information temporaire qu'on stocke dans la RAM (mémoire vive).

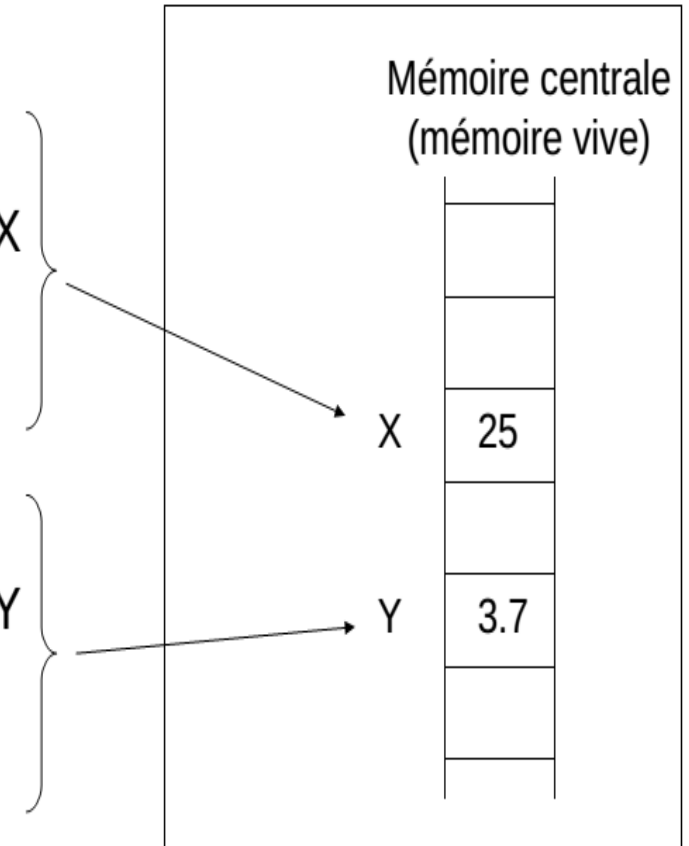
❑ On dit qu'elle est « variable » car c'est une valeur qui peut changer pendant le déroulement du programme.

Une variable possède :

- ❑ un **identificateur** : nom unique par lequel on peut accéder à ce contenu
- ❑ un **type** qui définit la taille de la place occupée
- ❑ une **valeur** contenue dans la case mémoire

- Identificateur : X
- Type : entier
- Valeur : 25

- Identificateur : Y
- Type : réel
- Valeur : 3.7



Variables locales vs. Globales – 1

Les variables locales

- ✓ Les variables déclarées dans un bloc d'instructions sont **uniquement visibles à l'intérieur de ce bloc**. On dit que ce sont **des variables locales** à ce bloc.
- ✓ Ces éléments sont uniquement accessibles depuis la fonction. On dit que ces variables sont **privées/locales**.

Exemple

```
void fonction_1()
{
    int a = 5;
}
void fonction_2()
{
    printf("%d\n", a);
}
```

Variables locales vs. Globales – 1

Les variables globales

- ✓ Les variables déclarées au **début du fichier**, à l'**extérieur** de toutes les fonctions sont disponibles à toutes les fonctions du programme. Ce sont alors des **variables globales**. En général, les variables globales sont déclarées immédiatement derrière les instructions **#include** au début du programme.
- ✓ **ATTENTION !!!!** Les variables déclarées au début de la fonction principale main ne sont pas des variables globales, mais elles sont locales à main !

Exemple

```
#include< stdio.h>
int a; // variable global
void fonction_1()
{
    a=5;
}
```

```
void fonction_2()
{
    printf("%d\n", a);
}
int main(void){
    fonction_1();
    fonction_2();
}
```

Passage par valeur

- ❑ Les paramètres peuvent être considérés comme des **variables locales** à la fonction.
- ❑ Lors de l'appel de la fonction, la valeur de chaque paramètre est copiée dans un nouvel espace mémoire réservé pour ce paramètre

=> Passage par valeur

```
# include <stdio.h>
int carre(int);

int main(void)
{
    int n, n2, n3;
    printf("Entrez un nombre ");
    scanf("%d", &n);
    n2 = carre(n);
    printf("Carre de %d = %d", n, n2);
    n3 = carre(n2);
    printf("Carre de %d = %d", n2, n3);
    return 0;
}

int carre (int x)
{
    int res;
    res = x * x;
    return res;
}
```

Prototype de fonction
type_r nom_funct (type_p, ...);

Appel à fonction
nom_funct (valeur p, ...);

Fonction retourne un valeur *int*

Fonction prends un valeur *int*
en parametre

Déclaration locale

Fonction retourne la valeur

Problématique

```
1  #include<stdio.h>
2
3  void incrementer(int x, int y){
4      x=x+1;
5      y=y+1;
6  }
7
8  int main(){
9      int n=3, m=4;
10     incrementer(n,m);
11     printf("n= %d et m= %d", n, m);
12     return 0;
13 }
```

Paramètres **formels** passés par valeur

Paramètres **effectifs**
(x copie de n)
(y copie de m)

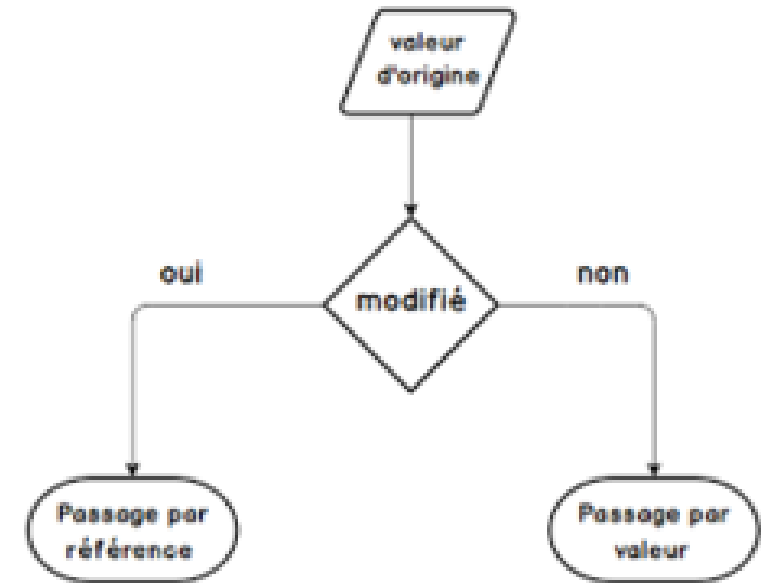
```
n= 3 et m= 4
```

```
-----
Process exited after 8.267 seconds with return value 0
Appuyez sur une touche pour continuer...
```

X et Y s'incrément mais aucun effet sur n et m

Solution

- ❑ Une fonction peut donc **localement** modifier le contenu de ses paramètres
- ❑ Si l'on veut qu'une fonction modifie **réellement** la valeur de ses paramètres, on ne passe pas la variable (en l'occurrence sa valeur) mais on doit passer son **adresse**.
- ❑ Le langage permet la transmission des adresses des variables en paramètres. Ceci s'appelle le **passage par adresse** (par référence dans d'autres langages de programmation).
- ❑ Comment réaliser ceci en langage C?
- ❑ **Les pointeurs sont la solution**



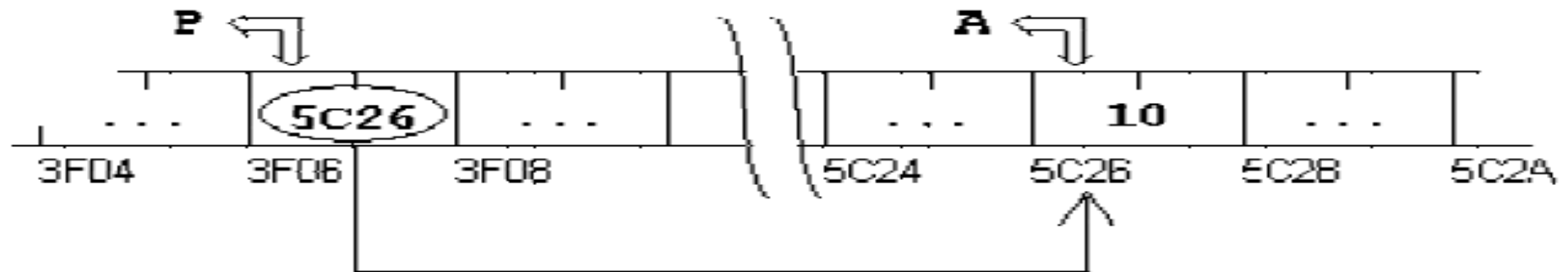
Les pointeurs

La notion de pointeurs

- ❑ Le nom d'une variable nous permet d'accéder **directement** à sa valeur. C'est ce qu'on appelle l'**adressage direct**.
- ❑ Si nous ne voulons pas utiliser le nom d'une variable, nous pouvons copier l'adresse de cette variable dans une **variable spéciale** appelée **pointeur**.
- ❑ L'utilisation de cette variable spéciale permet un accès indirect au contenu de la variable : **adressage indirect**

Soit **A** une variable contenant la valeur **10** et **P** un pointeur qui contient l'**adresse de A**.

En mémoire, **A** et **P** peuvent se présenter comme suit:



Définition

- ❑ Un pointeur est une **variable spéciale** qui peut contenir l'**adresse** d'une autre variable.
- ❑ Chaque pointeur est limité à **un type** de données.
- ❑ Si un pointeur P contient l'adresse de A, on dit que '**P pointe sur A**'

Remarque

- Un pointeur est une variable qui peut pointer sur différentes adresses.
- Le nom d'une variable reste toujours lié à la même adresse.

Opérateurs de base

- Lors du travail avec les pointeurs, nous avons besoin de 2 opérateurs principaux

Opérateur &

'Adresse de'

Pour obtenir l'adresse d'une variable.

Opérateur *

'contenu de'

Pour accéder au contenu d'une adresse (valeur).

- **&NomVariable** : fournit l'adresse de la variable <NomVariable>
- ***NomPointeur**: désigne le contenu de l'adresse référencée par le pointeur <NomPointeur>

Déclaration

- ❑ Pour déclarer un pointeur <NomPointeur> qui peut recevoir des adresses de variables de type <Type>

`Type *NomPointeur ;`

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int A=20;
```

```
    int *P; /*Déclaration d'un pointeur sur un entier*/
```

```
    P=&A;
```

```
    printf("le Contenu de la variable A est : %d",*P);
```

```
}
```

A : désigne le contenu de A

&A : désigne l'adresse de A

P : désigne l'adresse de A

***P : désigne le contenu de A**

Passage par adresse

- En revenant au problème de transmission de paramètre, on devra donc utiliser le **passage par adresse** si l'on veut qu'une fonction modifie la valeur d'une variable comme suit:

```
1  #include<stdio.h>
```

```
2
```

```
3  void incrementer(int *x, int *y){
```

```
4      *x=*x+1;
```

```
5      *y=*y+1;
```

```
6  }
```

```
7
```

```
8  int main(){
```

```
9      int n=3, m=4;
```

```
10     incrementer(&n,&m);
```

```
11     printf("n= %d et m= %d", n, m);
```

```
12     return 0;
```

```
13 }
```

Paramètres **formels** passés par adresse

Accès aux valeurs avec l'opérateur *

Paramètres effectifs appelés par l'opérateur &

```
n= 4 et m= 5
```

```
-----  
Process exited after 3.291 seconds with return value 0  
Appuyez sur une touche pour continuer...
```

Exemple récapitulatif

- ❑ Ecrire une fonction **Permut** qui permet de permuter les valeurs de 2 entiers, une fois par passage par valeur et une fois en utilisant le passage par adresse.

Passage par référence

```
# include <stdio.h>
void permut(int*, int*);
int main(void)
{
    int x = 3, y = 4;
    permut (&x, &y);
    printf("x = %d\n", x);
    printf("y = %d\n", y);
    return 0;
}

void permut(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

Passage par valeur

```
# include <stdio.h>
void permut(int, int);
int main(void)
{
    int x = 3, y = 4;
    permut (x, y);
    printf("x = %d\n", x);
    printf("y = %d\n", y);
    return 0;
}

void permut(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

Exercices

☐ **Exercice 1:** Quelle est l'erreur dans ce code?

```
int main()
{
    int * ptr; *ptr = 5 ;
    affichage (*ptr, '\n');
    return 0;
}
```

☐ **Exercice 2:** Quelle est la différence entre ?

☐ (*p1)++

☐ *(p1++)

☐ *p1++

☐ Que signifie *(&a)?

☐ **Exercice 3:** Compléter le tableau suivant

	a	b	c	p1	p2
initialisation	1	2	3	?	?
p1=&a;	1	2	3	&a	?
p2=&c;					
*p1=*p2;					
(*p2)++;					
p1=p2;					
p2=&b;					
*p1-=*p2;					
p1=*p2;					
a=*p2**p1;					
p1=&a;					
*p2=*p1/=*p2;					

Manipulation des pointeurs

Rappel sur les tableaux

- ❑ Les tableaux sont une suite de variables de **même type**, situées dans un espace **contigu** en mémoire.
- ❑ Il s'agit de "gros conteneurs " pouvant rassembler plusieurs valeurs du **même type désignées par un seul identificateur**
- ❑ La taille du tableau désigne le nombre d'éléments maximum qu'il pourra rassembler.
- ❑ Les éléments d'un tableau sont numérotés par des indices.
 - ❑ Un indice est toujours positif
 - ❑ L'indice du premier élément du tableau est 0.
 - ❑ L'indice du dernier élément du tableau est égal au nombre d'éléments -1

Adresse	Valeur
1600	10
1601	23
1602	505
1603	8

Tableaux et Adressage

- ❑ En C, il existe une relation très étroite entre pointeurs et tableaux.
- ❑ Chaque opération avec des indices de tableaux peut aussi être exprimée avec des pointeurs.
- ❑ Le nom d'un tableau représente l'adresse de son premier élément

$$\text{Nomtableau} = \&\text{NomTableau}[0] = \text{NomTableau} + 0$$

- ❑ Ainsi, si on considère un **tableau** **T** de type quelconque. On dit que la variable **T** est un pointeur **constant** (début du tableau) => **on ne peut pas modifier T**.
- ❑ La valeur **T[i]** (valeur de la ième composante) est évalué comme ***(T+i)**

```
1 #include<stdio.h>
2
3
4 int main(){
5     int tab[4];
6     //En utilisant le nom du tableau
7     printf("%d\n", tab);
8     //En utilisant l'adresse du premier élément
9     printf("%d\n", &tab[0]);
10    return 0;
11 }
```

```
6487568
6487568
-----
Process exited after 2.745 seconds with return value 0
Appuyez sur une touche pour continuer...
```

Tableaux et pointeurs – 1

- ❑ Au lieu d'utiliser le pointeur constant T, Nous pouvons également déclarer un pointeur de type du tableau pour pointer vers le tableau T.
- ❑ En déclarant un tableau T de type int et un pointeur P sur int, on a: `int T[10], int* P;`

`P = T;`  `P = &T[0];`

- ❑ En utilisant le même pointeur P, on pourra accéder à l'élément (la composante) suivant(e) avec l'instruction `P+1`.
- ❑ Le pointeur P pourra donc être utilisé pour accéder de manière **dynamique** à toute composante du tableau T.

Tableaux et pointeurs – 2

- ❑ En général si P pointe sur une composante d'un tableau, on a :
 - ❑ $P+i$ pointe sur la ième composante **après** P
 - ❑ $P-i$ pointe sur la ième composante **avant** P
 - ❑ $*(P+i)$ désigne le contenu de la ième composante **après** P
 - ❑ $*(P-i)$ désigne le contenu de la ième composante **avant** P

Remarque

Il faut bien faire attention avec ce genre d'opération à ne pas sortir du bloc mémoire => Le langage C n'effectue aucun test dans ce cas

Arithmétique des pointeurs

☐ Affectation

Soient P1 et P2 deux pointeurs sur le même type de données alors $P1 = P2$ signifie qu'on fait pointer P1 sur le même objet que P2

☐ Addition et soustraction

Si P pointe sur l'élément $T[i]$ d'un tableau alors:

- ☐ $P+n$ pointe sur $T[i+n]$

- ☐ $P-n$ pointe sur $T[i-n]$

☐ Incrémentation et décrémentation

Si P pointe sur l'élément $T[i]$ d'un tableau, alors après l'instruction :

- ☐ $P++$; P pointe sur $T[i+1]$

- ☐ $P+=n$; P pointe sur $T[i+n]$

- ☐ $P--$; P pointe sur $T[i-1]$

- ☐ $P-=n$; P pointe sur $T[i-n]$

Exemple

En utilisant un pointeur

```
1  #include <stdio.h>
2  int main(void){
3      int Tab[4] = {3, 2, 7, 9}, i;
4      int *p;
5      p = Tab;
6      for (i = 0; i < 4; i++){
7          printf("%d \n", *p);
8          p++;
9      }
10
11     return 0;
12 }
```

En utilisant un pointeur constant

```
1  #include <stdio.h>
2  int main(void){
3      int Tab[4] = {3, 2, 7, 9}, i;
4      int *p;
5      p = Tab;
6      for (i = 0; i < 4; i++){
7          printf("%d \n", *(Tab + i));
8      }
9
10     return 0;
11 }
```

Exercice 1

Soit P un pointeur qui 'pointe' sur un tableau A:

```
int A[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};  
int *P; P = A;
```

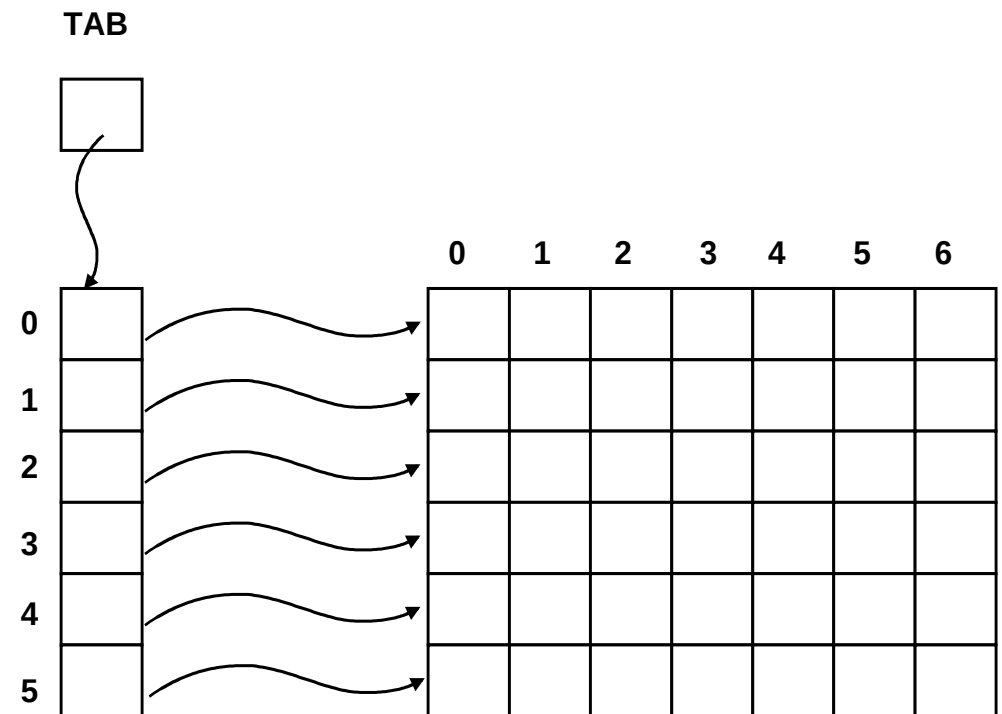
Quelles valeurs ou adresses fournissent ces expressions:

- a) `*P+2`
- b) `*(P+2)`
- c) `&P+1`
- d) `&A[4]-3`
- e) `A+3`
- f) `&A[7]-P`
- g) `P+(*P-10)`
- h) `*(P+*(P+8)-A[7])`

Tableau à 2 dimensions

- ❑ Un tableau 2d est un tableau de tableau de dimension 1
- ❑ On peut donc dire que le nom d'un tableau 2d est l'adresse d'un tableau d'adresse de tableaux de dimension 1
- ❑ On dit aussi qu'il s'agit d'un pointeur de pointeur

- exemple:
 - `int TAB[6][7];`
 - `int **p;` ➔ déclaration d'un pointeur de pointeur
 - `p = TAB;` ➔ initialisation du pointeur de pointeur



En résumé...

- si **B est un tableau** alors

- B désigne l'adresse de la première composante de B. (Cette adresse est toujours constante)

- B[i] désigne le contenu de la composante i du tableau

- &B[i] désigne l'adresse de la composante i du tableau en utilisant le formalisme pointeur:

- B+i désigne l'adresse de la composante i du tableau

- *(B+i) désigne le contenu de la composante i du tableau

- Si **P pointe** dans un **tableau**, alors

- P désigne l'adresse de la première composante

- P+i désigne l'adresse de la i-ième composante derrière P

- *(P+i) désigne le contenu de la i-ième composante derrière P

Allocation dynamique

Taille des variables

- ❑ Quand on déclare une variable dans un programme, on lui réserve implicitement un certain nombre d'octets en mémoire. Ce nombre est connu avant l'exécution du programme
- ❑ Selon le type de variable (char, int, double, float...), on a besoin de plus ou moins de mémoire. Exemple, un « char » tient sur 1 octet, un « int » sur 4 octets...
- ❑ Pour vérifier la capacité mémoire d'une variable, on dispose en C de la fonction:

`Sizeof(nom_variable)`

```
#include<stdio.h>
#include<stdlib.h>
int main() {
    printf("char : %d octets\n", sizeof(char));

    int T[3];
    printf("Le tableau T : %d octets\n", sizeof(T));

    return 0;
}
```

Allocation dynamique

- ❑ Dans des cas évidents, c'est le système d'exploitation qui se charge d'allouer (réserver) automatiquement l'espace mémoire suffisant pour une variable
- ❑ Dans d'autres cas, on peut le faire manuellement et demander de l'espace mémoire au système d'exploitation => **Allocation dynamique**
- ❑ La bibliothèque `<stdlib.h>` renferme plusieurs fonctions qui permettent de gérer l'allocation dynamique

Malloc et free

- ❑ On a besoin donc, d'inclure la bibliothèque <stdlib.h> qui contient 2 fonctions :
 - ❑ **malloc** ("Memory ALLOCation", c'est-à-dire "Allocation de mémoire") : demande au système d'exploitation la permission d'utiliser de la mémoire.
 - ❑ **free** ("Libérer") : permet d'indiquer au S.E. que l'on n'a plus besoin de la mémoire qu'on avait demandée. Après utilisation de la mémoire, celle-ci doit impérativement être libérée. Un autre programme peut maintenant s'en servir au besoin.
- ❑ La fonction **malloc**, qui alloue la mémoire, retourne l'adresse de l'emplacement mémoire alloué. On peut mémoriser cette adresse dans une variable pouvant contenir des adresses : **une variable de type pointeur**.

Exemple

- ❑ On va écrire un programme en C qui demande l'âge à l'utilisateur et qui va le lui afficher. La variable utilisée va être allouée manuellement (ou dynamiquement) au lieu d'automatiquement par le S.E.

```
#include<stdio.h>

#include<stdlib.h>

int main() {

    int *memoireAllouee = NULL;

    memoireAllouee = malloc(sizeof(int));

    if (memoireAllouee == NULL) exit(0);

    printf("Quel age avez-vous ? ");

    scanf("%d",memoireAllouee);

    printf("Vous avez %d ans\n",*memoireAllouee);

    free(memoireAllouee); }
```

Allocation dynamique et tableau

- ❑ La vraie utilité de l'allocation dynamique est pour créer un tableau dont on ne connaît pas la taille avant l'exécution du programme.
- ❑ **Exemple:** On veut écrire un programme qui stocke l'âge de tous les amis de l'utilisateur dans un tableau, on peut utiliser : `int ageAmis[15];`
- ❑ Le problème est que l'on ne sait pas à l'avance le nombre d'amis de l'utilisateur, on ne le saura qu'à l'exécution du programme.

```
int nombreAmis = 0, i = 0;

int* ageAmis = NULL;

printf("Combien d'amis avez-vous ? ");

scanf("%d", &nombreAmis);

if (nombreAmis > 0) {

    ageAmis = malloc(nombreAmis * sizeof(int));

    if (ageAmis == NULL){ exit(0);}

    for (i = 0 ; i < nombreAmis ; i++) {

        printf("Quel age a l'ami numero %d ? ", i + 1);

        scanf("%d", &ageAmis[i]); }

    for (i = 0 ; i < nombreAmis ; i++) {

        printf("%d ans\n", ageAmis[i]); }

    free(ageAmis); }
```

Allocation avec calloc

- ❑ La fonction **calloc**, comme la fonction malloc, alloue un emplacement mémoire et retourne l'adresse du premier octet.
- ❑ La fonction **calloc** prend deux paramètres, le nombre d'éléments et le nombre d'octets de chaque élément.
- ❑ La fonction **calloc**, contrairement à malloc, initialise tous les octets à 0.

malloc vs. calloc

malloc	calloc
Signifie allocation de la mémoire	Signifie allocation contiguë
prend 1 paramètre: le nombre d'octets	prend 2 paramètres: le nombre de blocs et la taille de chaque bloc
Syntaxe: Void* malloc(size_t n);	Syntaxe: Void* calloc(size_t n, size_t size);
Alloue n octets de mémoire. Si l'allocation réussit, renvoie un pointeur vide sur la zone mémoire allouée.	Alloue un bloc de mémoire contigu suffisamment grand pour contenir n éléments. La zone allouée est toute initialisée à 0.
Malloc est plus rapide	Calloc prend un peu plus de temps que malloc en raison de l'initialisation à 0.

Exemple

□ on alloue un tableau dont tous les éléments sont nuls sauf ceux rentrés par l'utilisateur

```
#include <stdio.h>
#include <stdlib.h>
float* RentrerTableau(int *addrNbreElements)
{
    int i, n; char choix;
    float *tab;
    printf("Entrez la taille du tableau : ");
    scanf("%d", &n);
    *addrNbreElements = n;

    tab = (float*)calloc(n, sizeof(float));
    puts("Entrez les éléments non nuls du tableau :");
    choix = 'y';
    while (choix=='y')
    { .....
    }
    return tab; /* on retourne l'adresse du tableau */ }

```

```
void Affichage(float *tab, int nb) {
    int i;
    for (i=0 ; i<nb ; i++)
        printf("tab[%d] = %f\n", i, tab[i]);
}

int main(void) {
    int nb;
    float *tab;
    tab = RentrerTableau(&nb);
    Affichage(tab, nb);
    free(tab) /* libération de mémoire obligatoire */
    return 0;
}

```

realloc

- ❑ La méthode “realloc” ou “réallocation” est utilisée pour modifier dynamiquement l'allocation mémoire d'une mémoire allouée précédemment.
- ❑ En d'autres termes, si la mémoire allouée précédemment à l'aide de malloc ou calloc est insuffisante, realloc peut être utilisé pour réaffecter dynamiquement de la mémoire.

```
ptr = realloc(ptr, newSize);
```

- ❑ ptr est réaffecté avec la nouvelle taille 'newSize'.

Exemple

```
#include <stdio.h>
#include <stdlib.h>
int main(void){
    int *tab;
    int i, n;
    tab=(int*)calloc(5, sizeof(int));
    if (tab == NULL) {
        printf("Mémoire non allouée.\n");
        exit(0);
    }
    else{
        printf("Mémoire allouée avec succès \n");
        for(i=0 ; i < 5 ; i++){
            *(tab+i)=i;
        }
        printf("Les éléments du tableau sont: ");
        for(i=0 ; i < 5 ; i++){
            printf("%d, ", *(tab+i));
        }
    }
}
```

```
// nouvelle taille
n=10;
tab=realloc(tab, n * sizeof(int));

for(i=5 ; i < n ; i++){
    *(tab+i)=i;
}
printf("Les éléments du tableau après \"réallocation\" ");
for(i=0 ; i < n ; i++){
    printf("%d, ", *(tab+i));
}

free(tab);
}
return 0;
}
```

Exercices

Exercice 1

- ☐ Ecrire la fonction qui alloue la mémoire d'un vecteur de taille `dimension`, puis qui l'initialise à la valeur `val`, « `int * alloue_vecteur(int dimension, int val)` ».
- ☐ Ecrire la fonction « `void libere_vecteur(int * vecteur)` », qui libère le vecteur `vecteur`.
- ☐ Afficher ce vecteur pour tester vos fonctions

Exercice 2

- ☐ Ecrire la fonction qui alloue la mémoire d'une matrice de taille « `lignes*colonnes` », puis qui l'initialise à la valeur `val`, « `int ** alloue_matrice(int t lignes, int colonnes, int val)` ».
- ☐ Ecrire la fonction « `void libere_matrice(int ** matrice, int n)` », qui libère la matrice `matrice`.
- ☐ Afficher cette matrice pour tester vos fonctions.

Correction

Exercice 2

Exercice 1

```
int * alloue_vecteur(int dimension, int val)
{
    int i;
    int * vecteur = malloc(dimension*sizeof(int));
    for(i=0;i<dimension;i++)
        vecteur[i]=val;
    return vecteur;
}

void libere_vecteur(int * vecteur)
{
    free(vecteur);
}
```

```
int ** alloue_matrice (int lignes, int colonnes,int val)
{
    int i,j;
    int** matrice = malloc(lignes*sizeof(int *));
    for (i=0;i<lignes;i++)
        matrice[i]= malloc(colonnes*sizeof(int));
    for (i=0;i<lignes;i++)
        for(j=0;j<colonnes;j++)
            matrice[i][j]=val;
    return matrice;
}

void libere_matrice(int ** matrice,int lignes)
{
    int i;
    for (i=0;i<lignes;i++)
        free((int *)matrice[i]);
}
```

Fin du chapitre 1
